

Intelligent Dead Reckoning (IDR) — Final Planning & Implementation Plan

Unified Build, With Polished UI/UX Track (Jetpack Compose + Animations)

This supersedes the earlier plan's Track B. All four tracks still start on Day 1, organized by real dependency, not by competition stage. Track B is now scoped for a genuinely polished app, not a functional-only prototype — which adds real time, documented honestly below.

1. The Four Tracks (Updated)

Track	What It Builds	Change From Before
A — AI/Data Science	Dataset, INS baseline, AI model, export	Unchanged
B — App Development	Full Android app: Jetpack Compose , animated map, polished UI/UX	Expanded — Compose learning, design pass, specific animated components
C — Algorithm Core	Calibration, EKF fusion, map matching	Unchanged
D — Edge Engine	Higher-rate scaffold, shared core integration	Unchanged

2. Track B — Full Detail (Updated)

B1. Environment Setup (1 day) — Install Android Studio; confirm a project runs on emulator/device.

B2. Kotlin Basics (1–2 weeks) — Variables, functions, conditionals, loops, basic classes (Google's free Android Basics course).

B2.5 — NEW: Jetpack Compose Fundamentals (1–2 weeks)

Composable functions, state hoisting, recomposition, `remember`/`mutableStateOf`.

Compose's animation APIs specifically: `animateFloatAsState`, `AnimatedVisibility`, `Crossfade`, `updateTransition`.

Why this is necessary, not optional: Compose's reactive model matches exactly what this app needs — continuously reacting to a stream of position/state updates with smooth visual transitions. Building this

with the legacy View system would mean fighting the framework the whole way.

B3. Practice App 1 — UI Fundamentals (Compose) (1-2 days) — A button that animates a state change, built in Compose from the start.

B4. Practice App 2 — Sensor Reading (2-3 days) — `SensorManager` reading accelerometer/gyroscope/magnetometer, displayed with a simple animated value transition (not static text) — this is where you first practice the "reactive UI" pattern you'll use everywhere later.

B5. Practice App 3 — Map Display (3-4 days) — Google Maps SDK or `osmdroid`, one marker, practice moving it with an **eased animation** rather than an instant jump — this is the core interaction pattern the whole app depends on.

B6. — NEW: Lightweight Design Pass (3-5 days)

Rough wireframes/mockups (Figma, free tier, or even hand-sketched) for: main map screen, calibration flow, mode-status indicator, settings/trip-summary screens.

Purpose: avoid designing screens live in code — decide the look and flow first, build second.

B7. App Shell Assembly (1-1.5 weeks)

Combine sensor reading + animated map + navigation between screens (Compose Navigation with transition animations).

Build with placeholder/dummy position data — real data isn't needed yet.

B8. — NEW: Core Animated Components (2-3 weeks, the main polish work)

Smooth marker movement: eased interpolation between position updates (not linear — a proper easing curve).

Camera follow + rotate: smooth pan/rotate tracking vehicle heading.

Live status indicator: animated transitions between GNSS Available/Degraded/Denied visual states — built now with placeholder state changes, **tuned for real timing once Track C exists** (see Section 4).

Drift/confidence circle: animated radius around the vehicle icon.

Guided calibration flow UI: progress animation for the calibration sequence.

Screen transitions, Material 3 theming, dark/day-night mode, haptic feedback hooks.

B9. TensorFlow Lite Integration Tutorial (2-3 days) — Load a `.tflite` file, feed input, read a prediction, using any placeholder model first.

B10. Final Wiring (part of Phase 6 convergence, see Section 5) — Replace all placeholder data/models with Track A's real exported model and Track C's real fusion core.

3. Updated Timeline

Week	Track A	Track B	Track C	Track D
1	A1: Dataset verify	B1-B2: Setup, Kotlin	C1: Calibration start	D1: Scaffold

Week	Track A	Track B	Track C	Track D
2	A2: INS baseline	B2 cont., B2.5: Compose start	C1 cont., C2: EKF start	D2: Ingestion loop
3	A3: AI model training	B2.5 cont.	C2: EKF continues	D2 cont.
4	A3 continues	B3–B4: Practice apps	C2: EKF continues	—
5	A3 finishing, A4: Export	B5: Map practice app	C3: Swap in AI input	—
6	(Screening evidence ready)	B6: Design pass	C4: AI-enhanced fusion	—
7	Support C4/C5	B7: App shell assembly	C4 cont.	D3: Plug in Track C core
8	—	B7 cont., B9: TFLite tutorial	C5: Map matching	D3 cont.
9–11	—	B8: Animated components build & tune	C5 cont., C6: Mode-switch validation	—
12–14	—	Phase 6: Full integration	Phase 6	Phase 6
15–16	—	Phase 7: Benchmark testing, iteration		
17	—	Phase 8: Field test, demo prep		

New total: ~16–17 weeks (vs. ~13–14 previously) — the added time is entirely in Track B’s Compose learning + design pass + dedicated animation-tuning work (B2.5, B6, B8), roughly the +2 to 4 weeks flagged earlier.

4. One Sequencing Note That Still Applies

Track B’s animated *mechanics* (marker easing, camera movement, screen transitions) can be built and practiced now, with fake data. But **final tuning of the mode-status animation specifically should wait until Track C’s real fusion engine exists** — animating a state transition that doesn’t exist yet risks needing rework once you see how the real system actually behaves (e.g., how often it flickers between Degraded/Denied, which affects whether the animation needs debouncing). This is a targeted wait on tuning, not a block on building.

5. Convergence Phases (Updated)

Phase 6 — Full Integration (3 weeks, was 2-3) — wire Track A's model + Track C's core into Track B's app; tune the mode-status and drift-circle animations against real fusion output; wire Track C's core into Track D's edge scaffold.

Phase 7 — Benchmark Testing (2-3 weeks, *iterative*) — unchanged from before: simulate GNSS-denied stretches, measure drift, iterate on model/filter tuning if the 10% target isn't met.

Phase 8 — Field Test & Demo Prep (~1 week) — real drive through a tunnel/underground area; final demo polish, since the UI is now a genuine presentation asset, not just a functional check.

6. Master Checklist (Updated)

Track A: dataset verified → INS baseline plotted → AI model trained → model exported.

Track B: Android Studio + Kotlin learned → **Compose fundamentals learned** → 3 practice apps built → **design pass completed** → app shell assembled → **animated components built** (marker easing, camera follow, status indicator, drift circle, calibration flow, transitions/theming) → TFLite tutorial done.

Track C: calibration engine → EKF fusion (raw INS, then AI-corrected input) → AI-enhanced fusion → map matching → mode-switch validated.

Track D: scaffold → ingestion loop → Track C core plugged in.

Convergence: full integration (real model + real fusion + tuned animations) → benchmark passed → field test → demo-ready.

7. Immediate Next Actions (Unchanged Starting Points, One Addition)

Track A: Download IO-VNBD, open one file in Python.

Track B: Install Android Studio, start Kotlin Basics — **and bookmark Jetpack Compose's official "Animations in Compose" documentation now**, since it's the throughline for everything B8 will build later.

Track C: Sketch calibration logic once A1 confirms the schema.

Track D: Set up the Python project skeleton and a mock 200Hz data generator.